

K-Means Clustering

How the algorithm groups similar points, how we choose the number of groups, and how to avoid a bad start.

Three Stops in This Session

01

K-Means geometric intuition

What it does, and the three steps it repeats

02

How to find the K value

WCSS, the elbow method, Euclidean and Manhattan distance

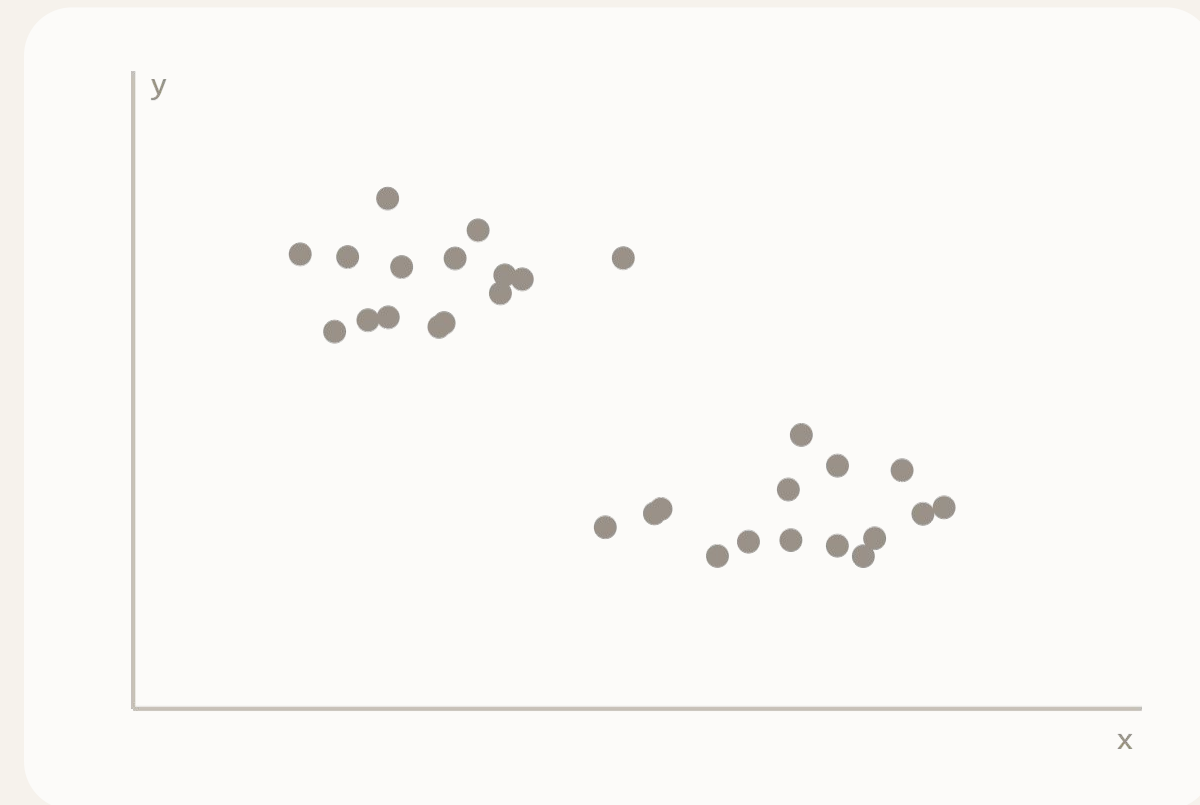
03

Random initialisation trap

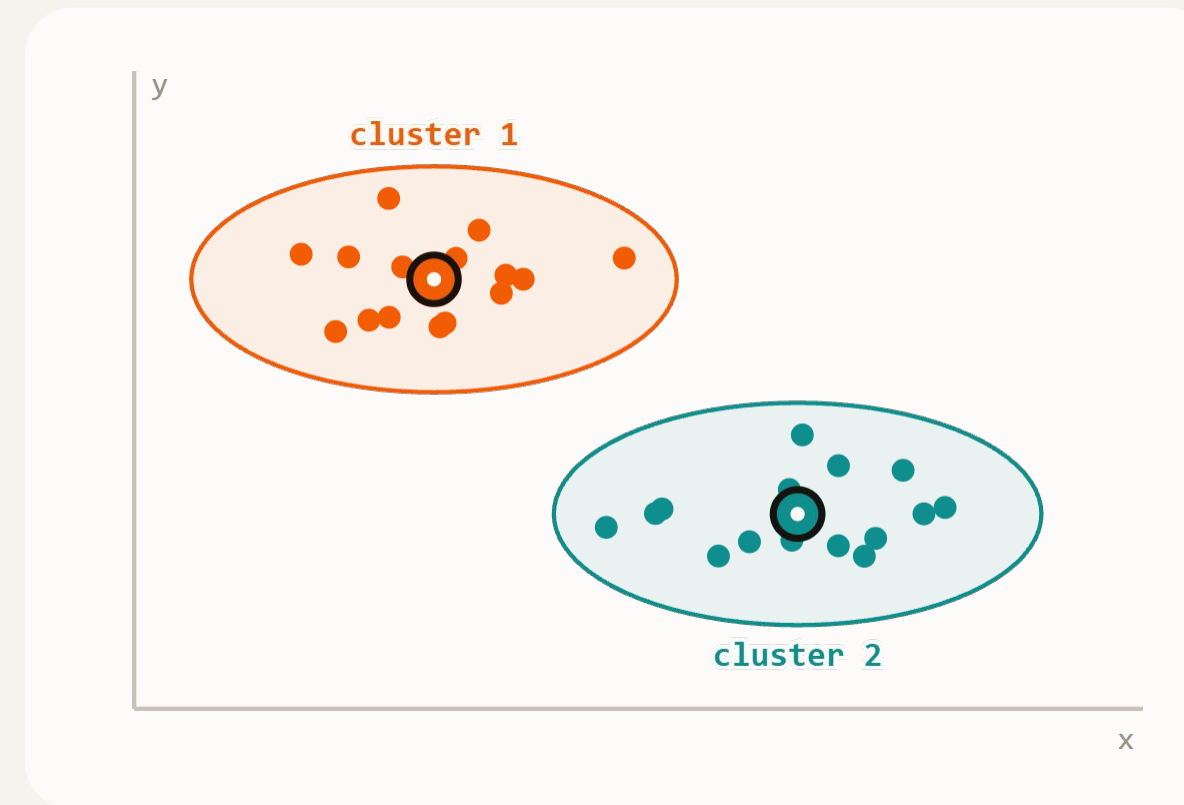
What goes wrong with a bad start, and how K-Means++ fixes it

Put Similar Points Into the Same Group

before: 30 points, no labels



after K-Means with $K = 2$



Cluster: a group of points that are close to each other.

Centroid: the centre point of a cluster. The big dot with the dark ring.

2 clusters give 2 centroids.

K Is the Number of Clusters

before: 36 points



after K-Means with $K = 3$



K = how many clusters we ask for = how many centroids.

Means = each centroid is the average of its points. We see this in step 3.

Goal: similar points end up together.

K-Means Repeats Three Simple Steps

1

Place K centroids

Put K centroids somewhere in the data, usually at random.

2

Join the nearest centroid

Every point joins the centroid closest to it. That makes the groups.

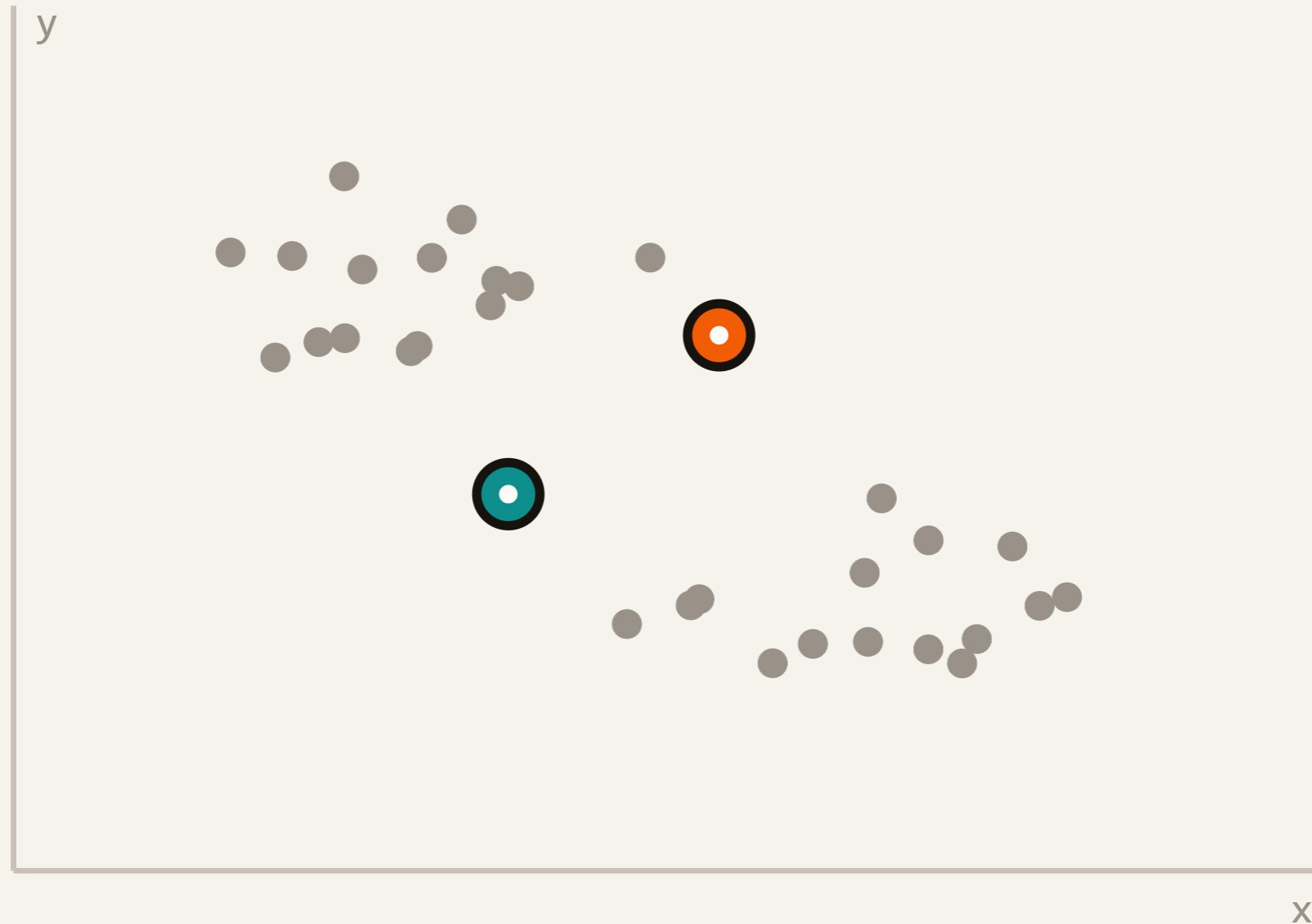
3

Move to the average

Move each centroid to the average of the points in its group.

↺ Repeat steps 2 and 3 until no point changes its group. Then stop.

Place $K = 2$ Centroids at Random



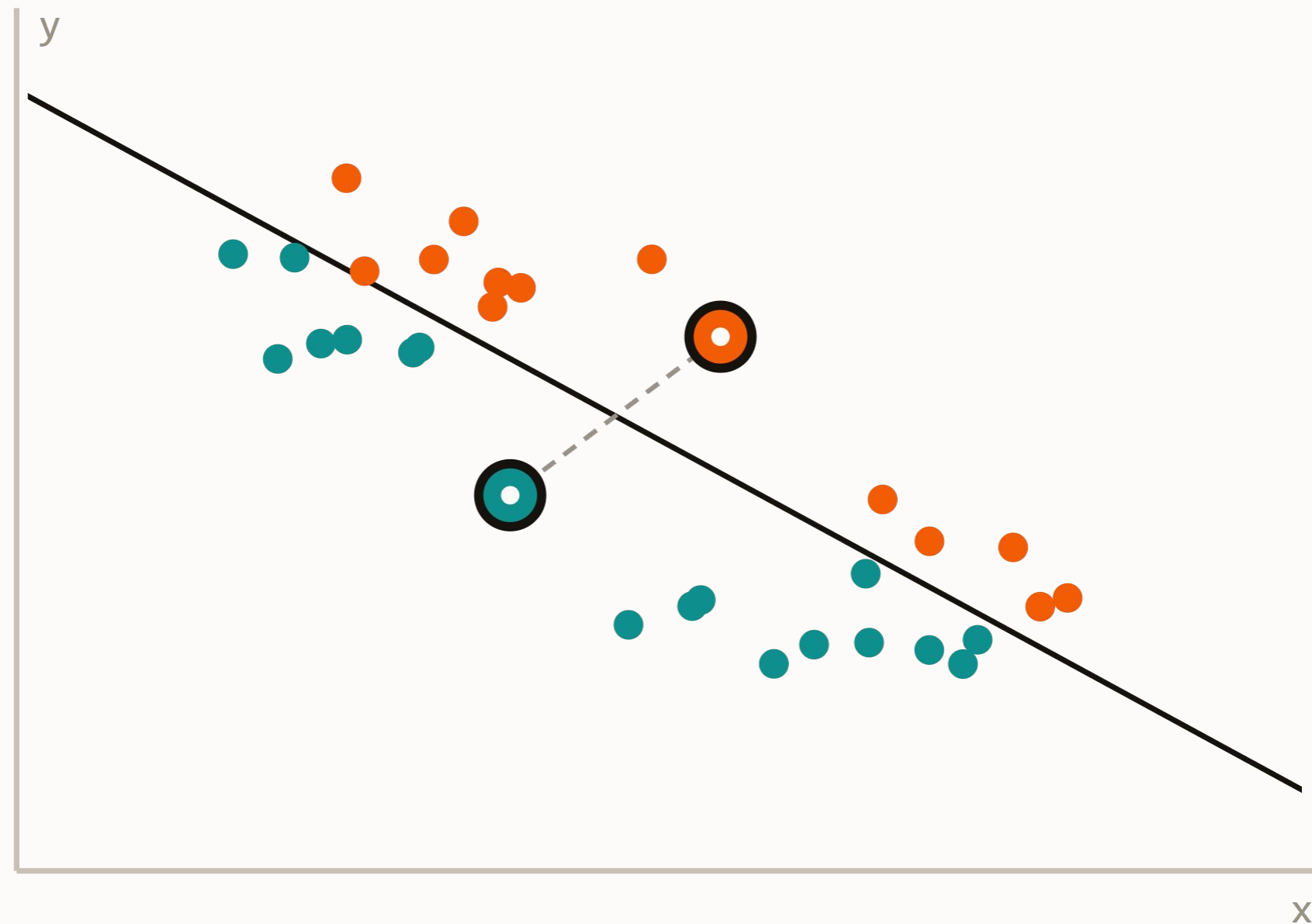
We choose $K = 2$, because we can see two groups. Choosing K properly comes in topic 2.

Two centroids are dropped at **random** spots. The orange one and the teal one.

Starting positions: orange (5.60, 6.60), teal (3.90, 4.60).

They are in bad spots. That is fine: the next steps will move them.

Every Point Joins Its Nearest Centroid



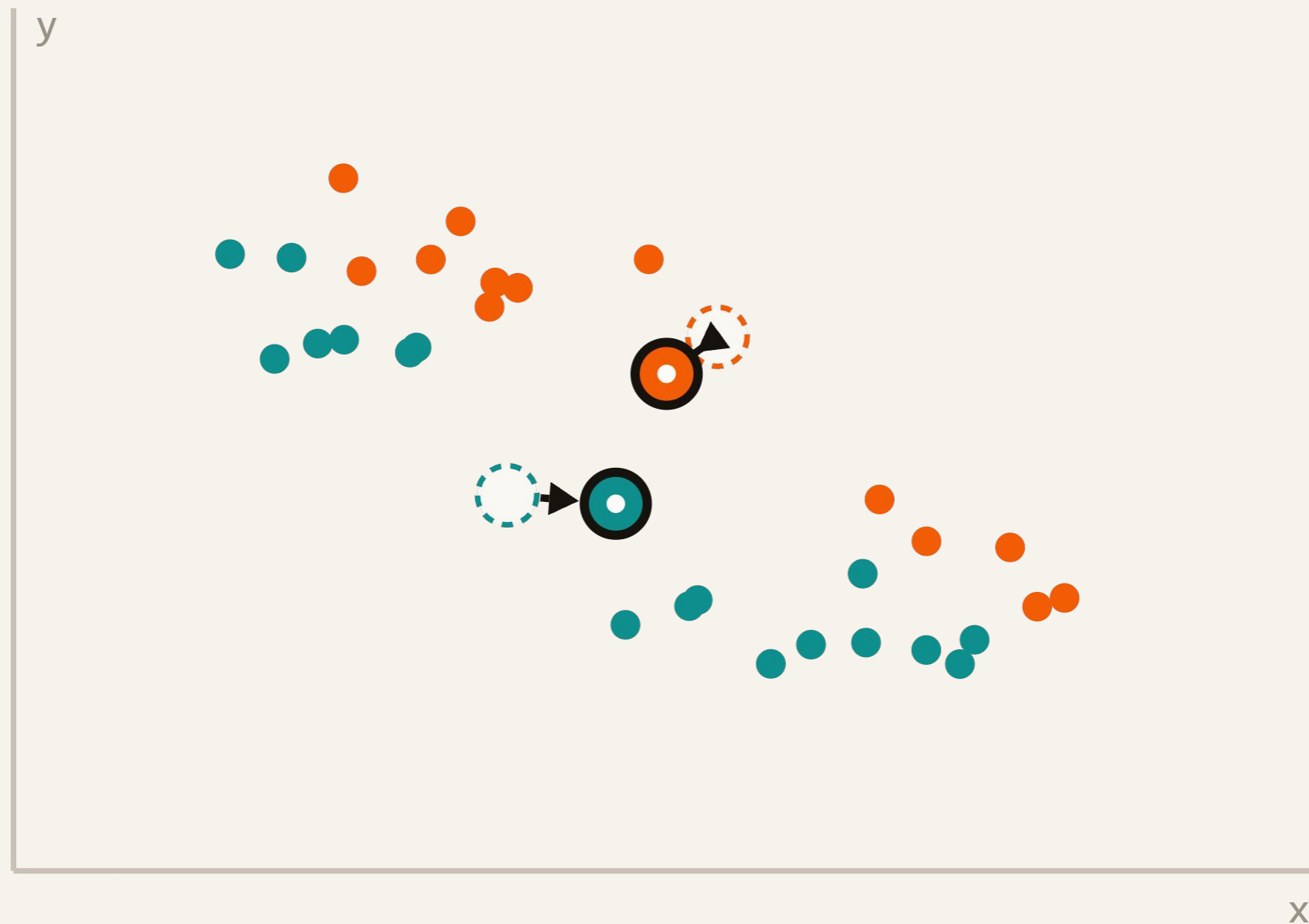
For each point, measure the distance to both centroids. The point joins the **closer** one.

Shortcut to see it: the **dashed line** joins the two centroids. The **solid line** cuts it in half at 90°.

One side of the solid line is closer to orange, the other side is closer to teal.

Distance is measured with **Euclidean** or **Manhattan** distance. Both come in topic 2.

Move Each Centroid to the Average of Its Points



ORANGE GROUP: 13 POINTS

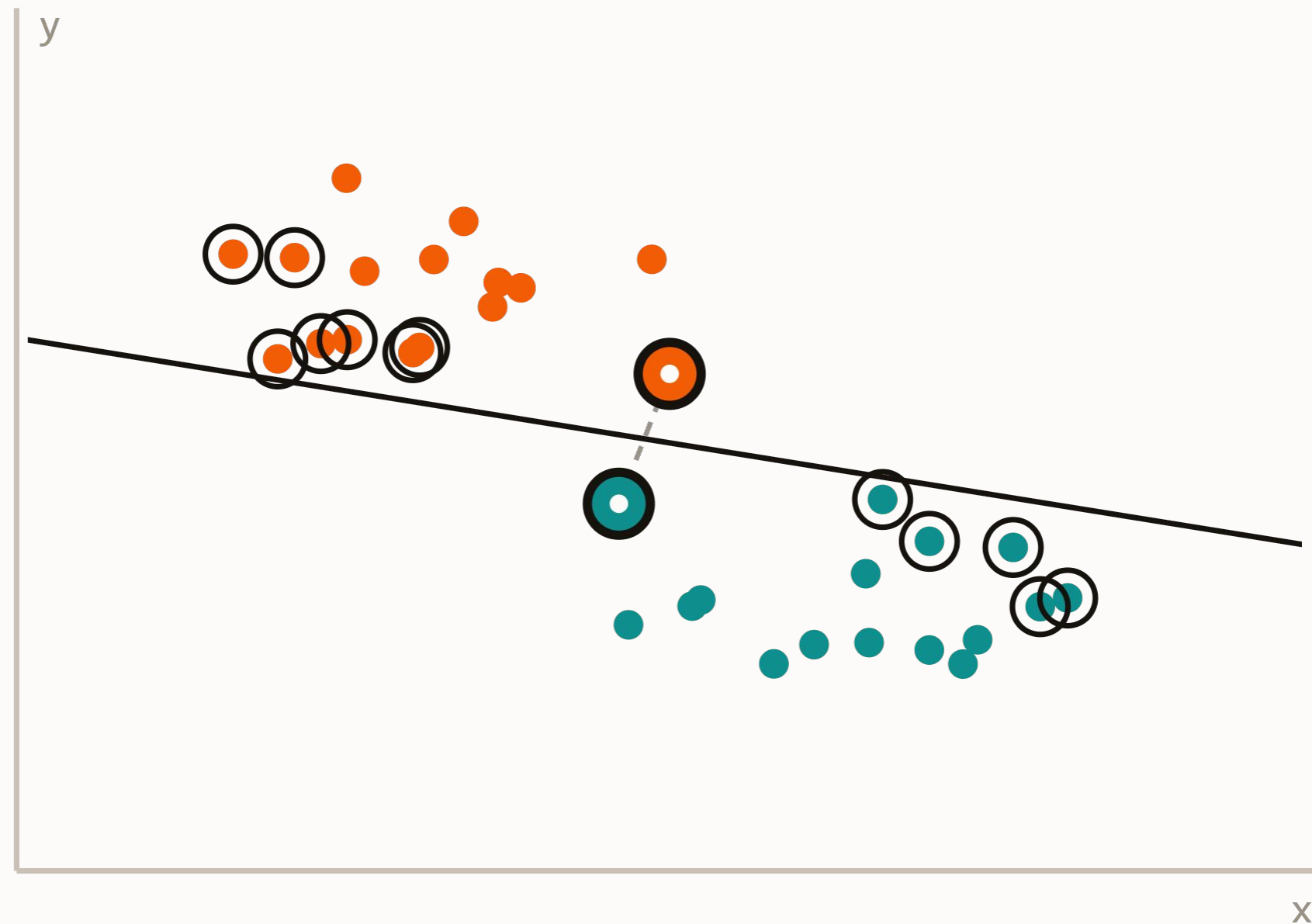
average $x = 67.46 / 13 = 5.19$ average $y = 79.71 / 13 = 6.13$

TEAL GROUP: 17 POINTS

average $x = 81.23 / 17 = 4.78$ average $y = 76.37 / 17 = 4.49$

Dashed circle = old position. Arrow = where it moved. This average is the **Means** in K-Means.

Repeat: Some Points Switch Groups



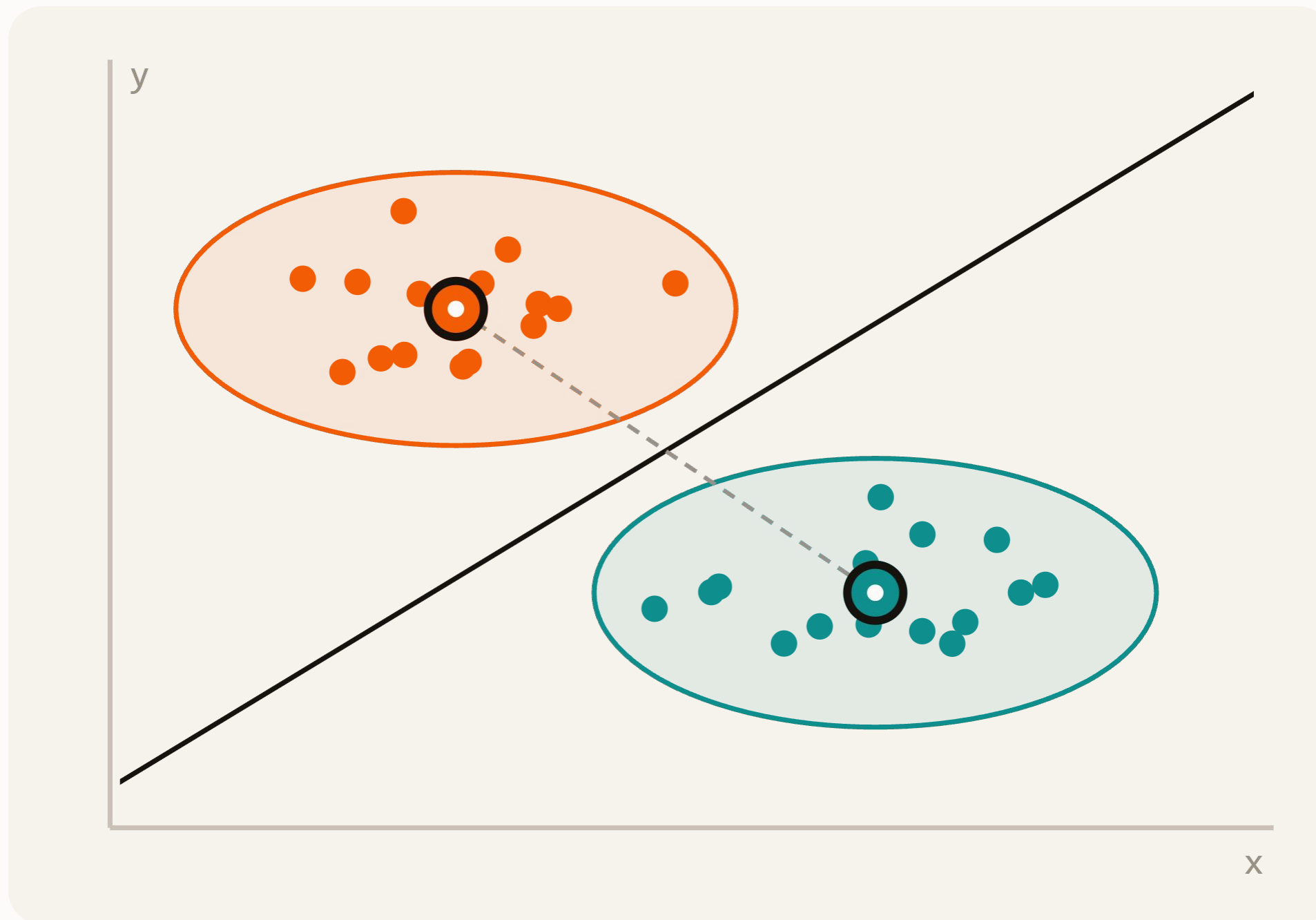
The centroids have moved, so we do **step 2 again** with the new positions.

The **12 ringed points** are now closer to the other centroid, so they switch groups.

Then **step 3 again** : move each centroid to the average of its new group.

Step 2, step 3, step 2, step 3... round and round.

Stop When No Point Changes Its Group



After a round of step 2, **no point changed** its group. So the averages stay the same and the centroids stop moving.

On this data it took **3 rounds**.

Final centroids: orange (3.05, 7.21), teal (6.86, 3.19).

Two clean clusters. That is K-Means.

Real Data Does Not Show Its Groups



Earlier we picked $K = 2$ just by looking. That only worked because the groups were clear.

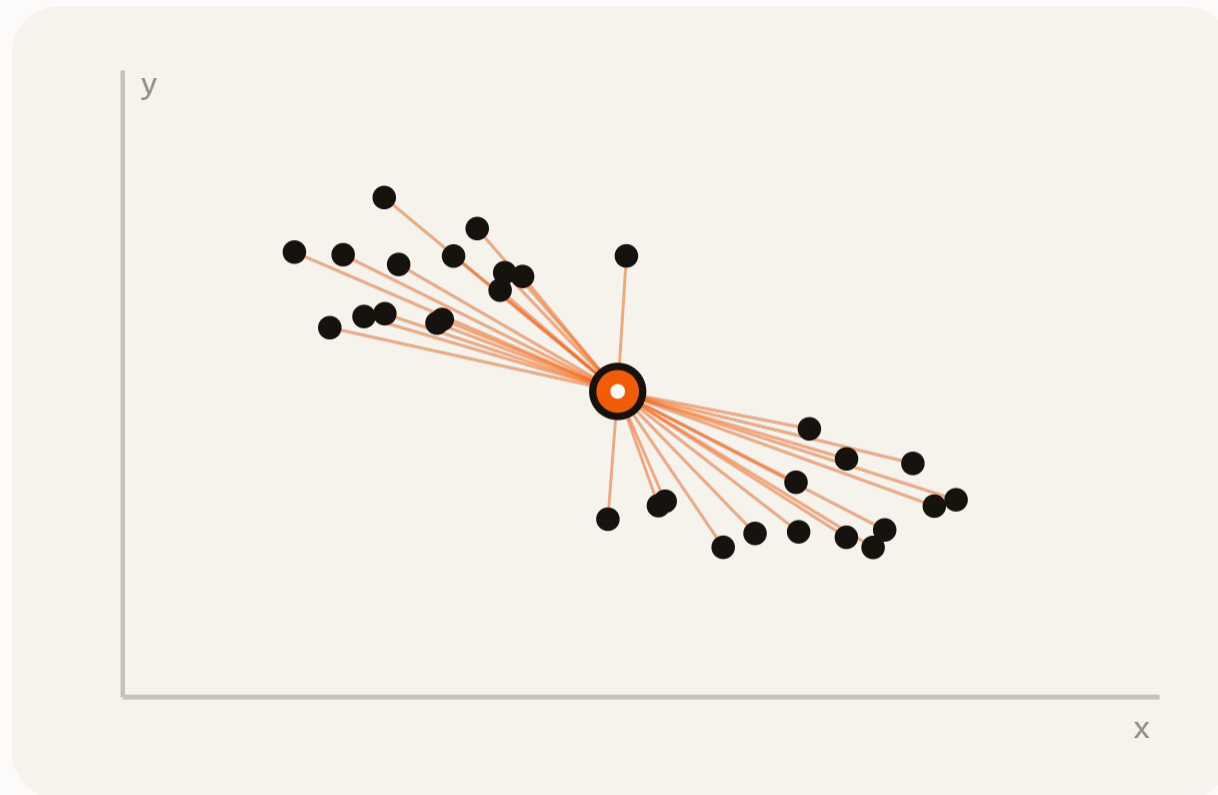
Here the points **overlap**. Is it 2 groups? 3? 4?

Real datasets have many features, so we cannot even draw them.

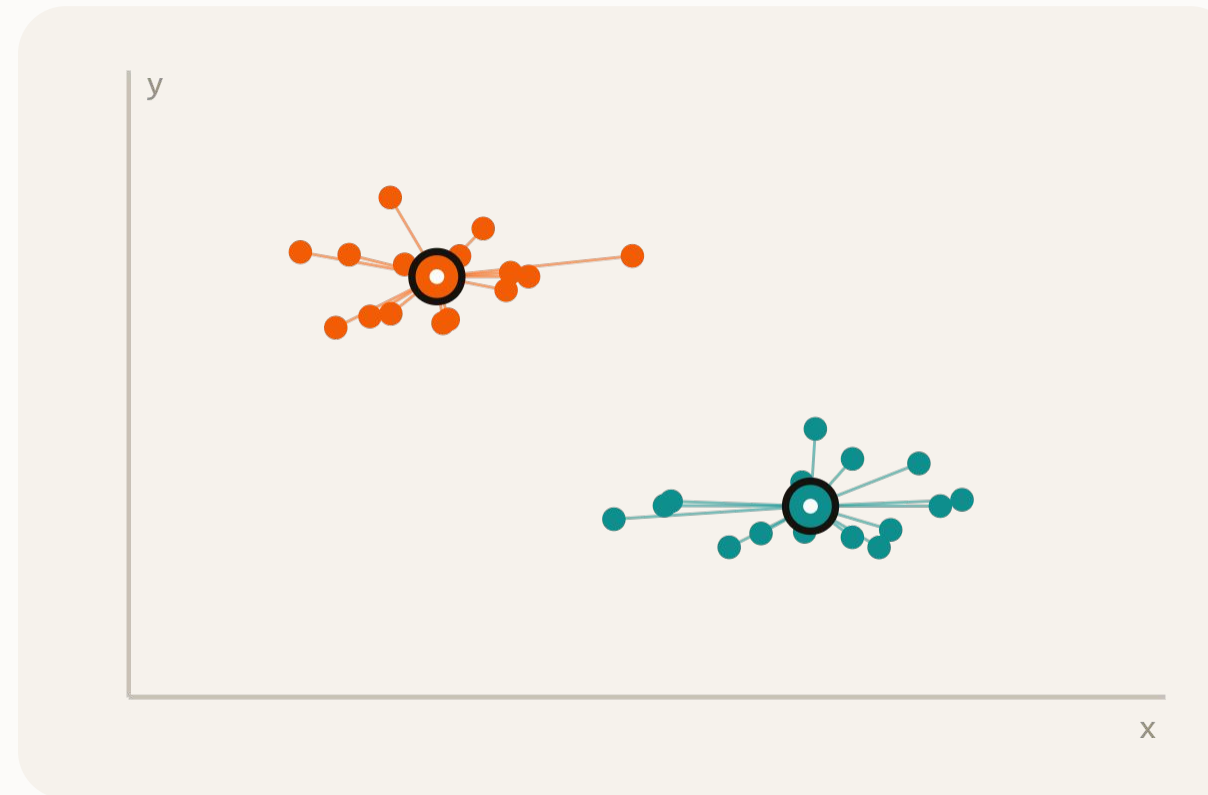
We need a number that scores each K . It is called **WCSS**.

WCSS: Within Cluster Sum of Squares

K = 1 · WCSS = 268.8



K = 2 · WCSS = 38.7



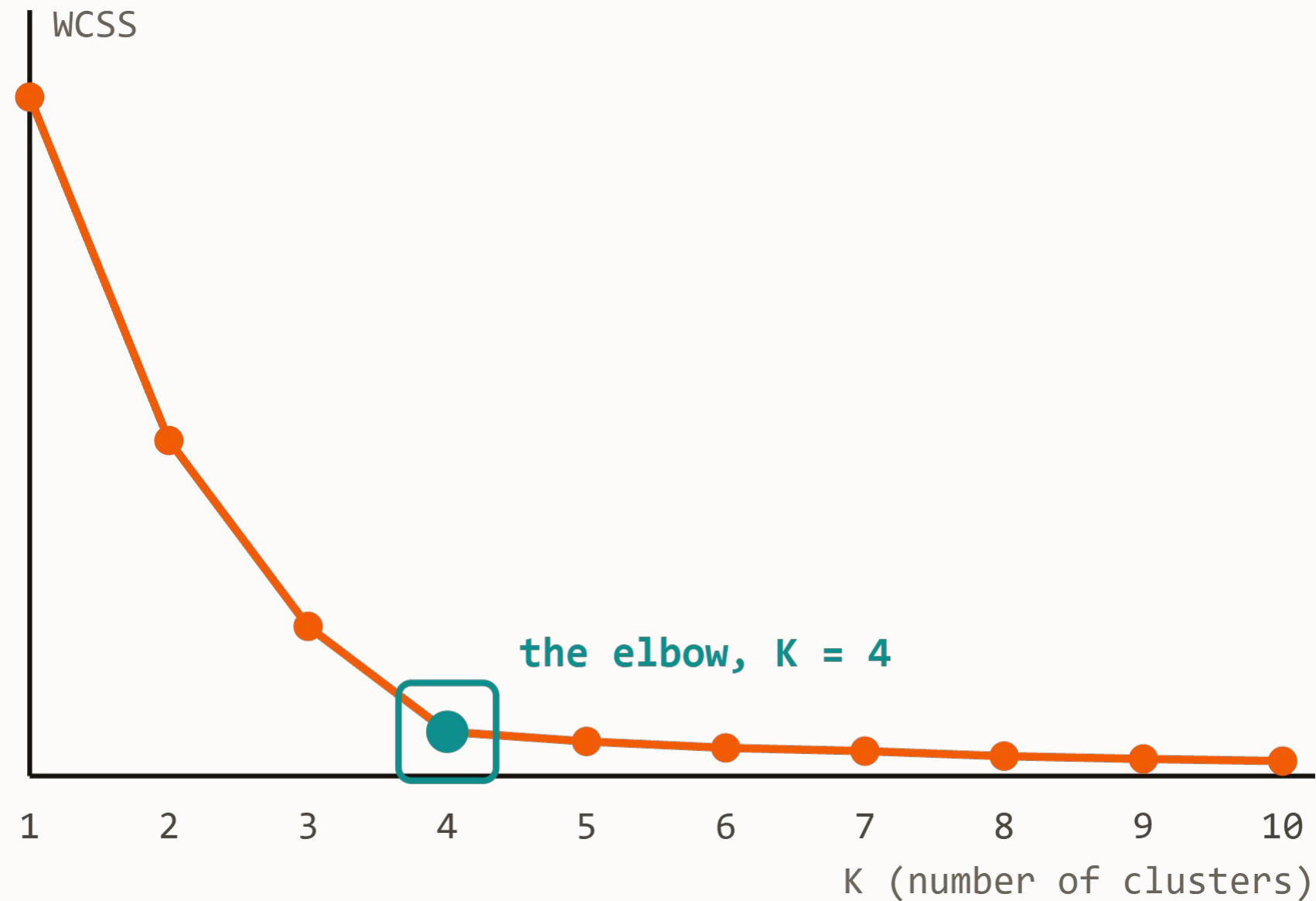
$$\text{WCSS} = \sum (\text{distance})^2$$

distance from each point to its nearest centroid, squared, then all added up

K = 1: long lines, big WCSS.

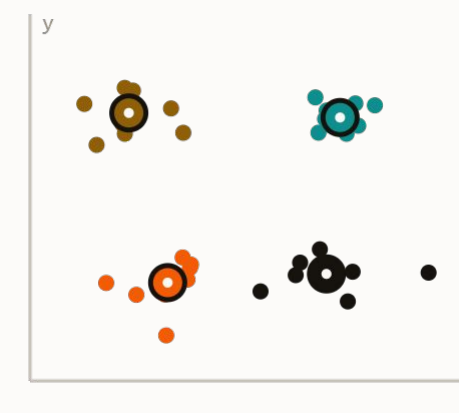
K = 2: short lines, small WCSS.

The Elbow Method Picks K



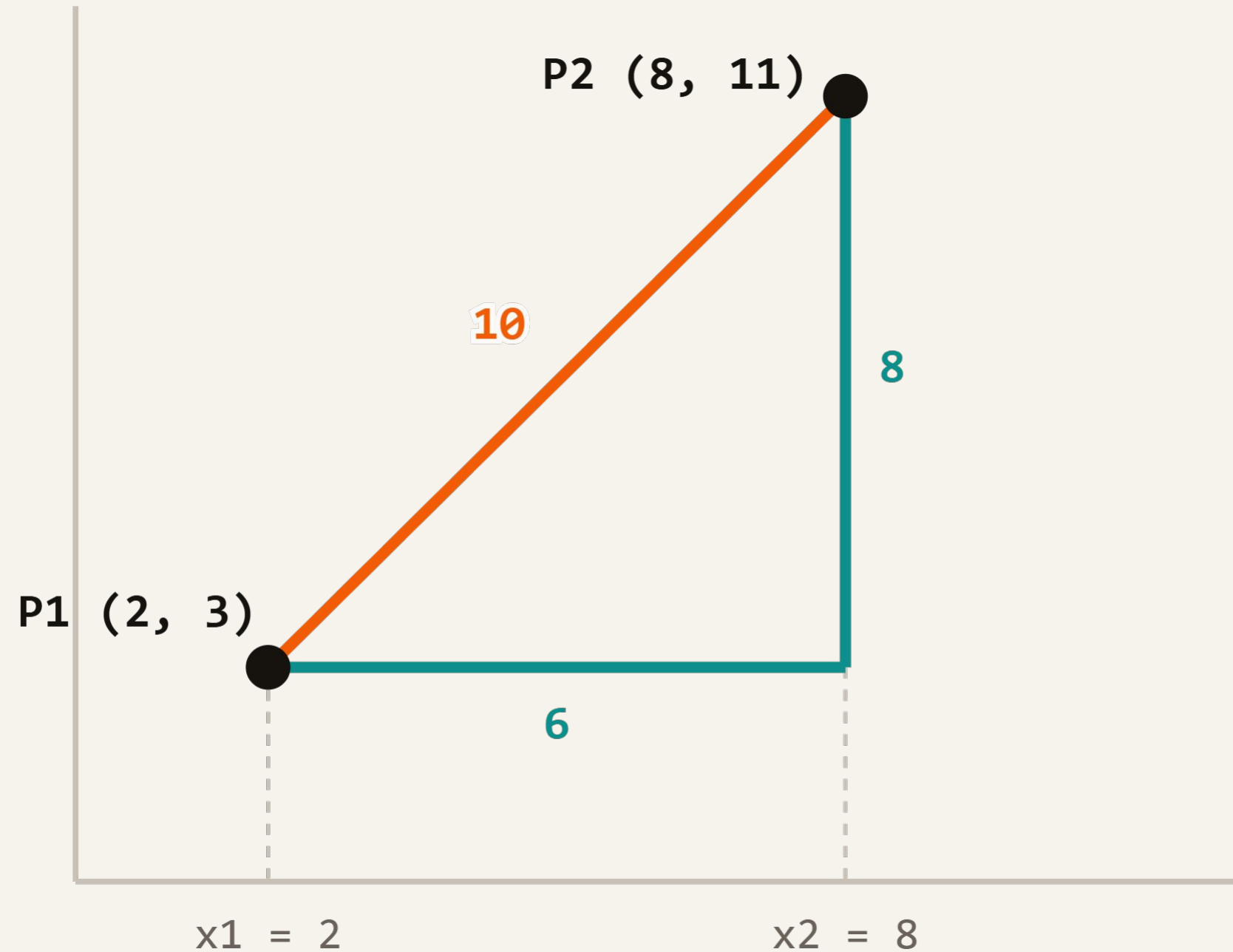
Run K-Means for $K = 1, 2, 3 \dots$ up to about 20. Note the WCSS each time.

Big drops until $K = 4$ (107.6 to 31.8). After that, tiny drops (31.8 to 24.9).



Pick K at the elbow, where the curve bends and goes flat. Here $K = 4$.

Euclidean and Manhattan Distance



EUCLIDEAN: STRAIGHT LINE

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$
$$= \sqrt{6^2 + 8^2} = \sqrt{100} = 10$$

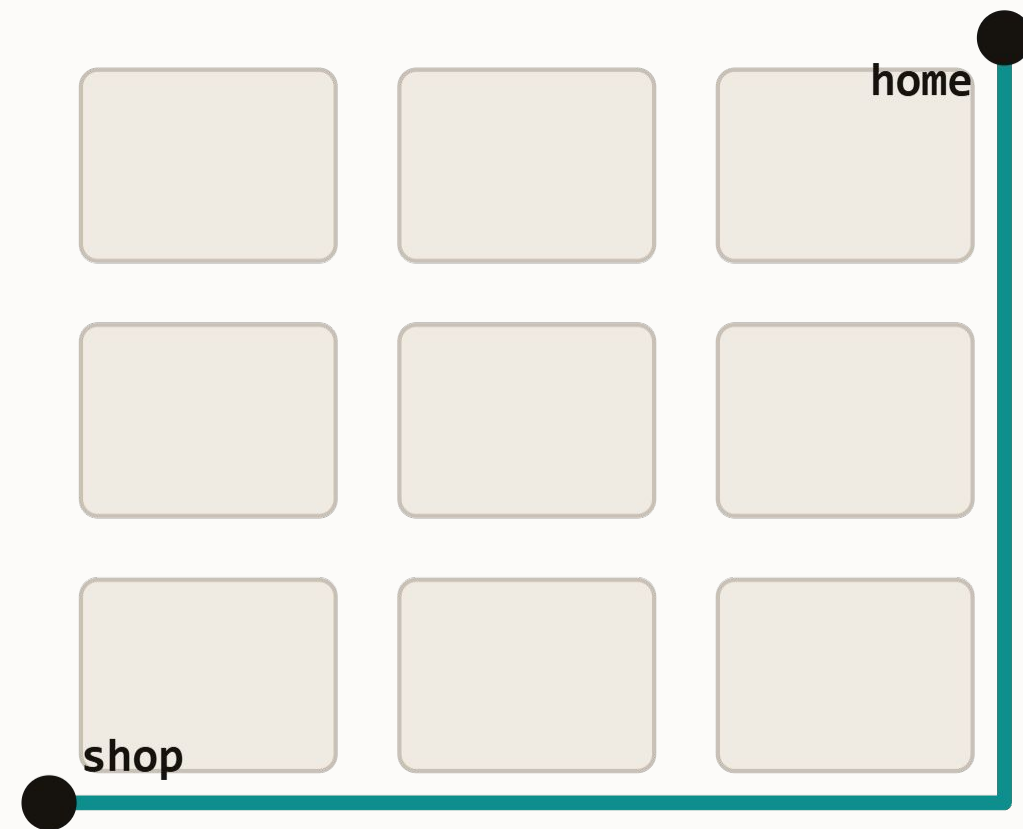
MANHATTAN: ALONG THE GRID

$$|x_2 - x_1| + |y_2 - y_1|$$
$$= 6 + 8 = 14$$

With 3 features, add a $(z_2 - z_1)$ part in the same way.

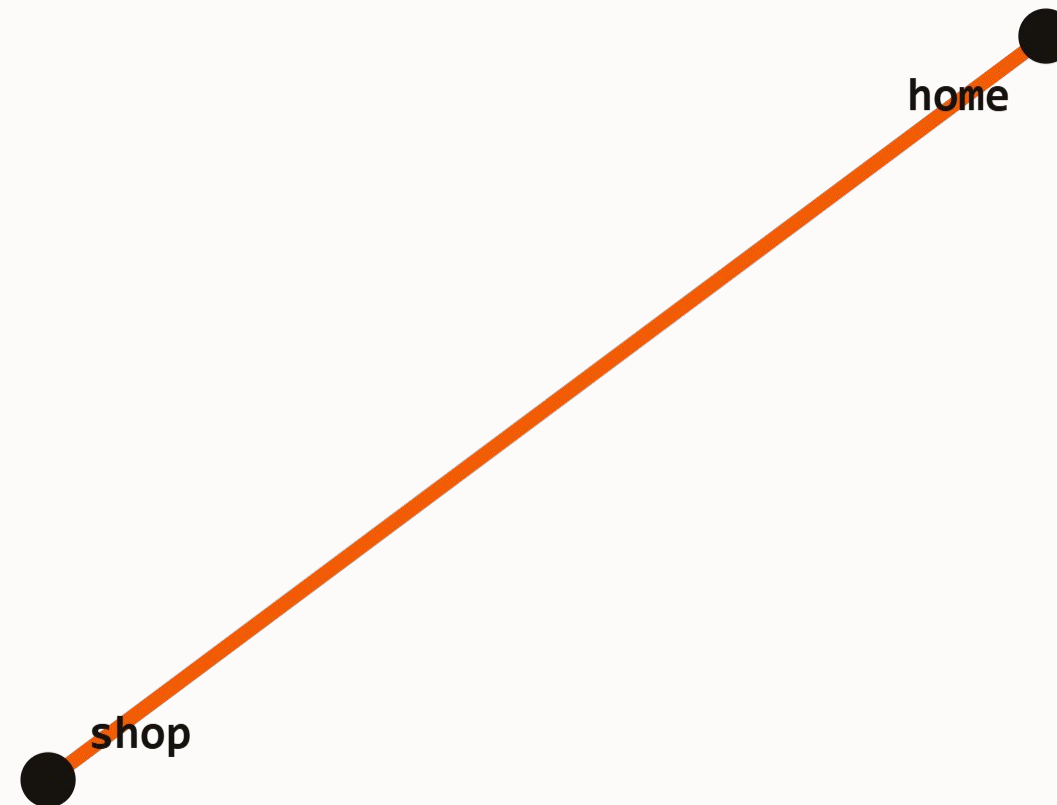
Roads or Straight Through the Air

DELIVERY BIKE IN A CITY OF BLOCKS



It cannot go through buildings, only along the roads. Use **Manhattan distance**.

DELIVERY DRONE



It flies in a straight line over everything. Use **Euclidean distance**.

K-Means normally uses **Euclidean** distance.

It is what scikit-learn KMeans uses.

We Expect Three Clusters Here



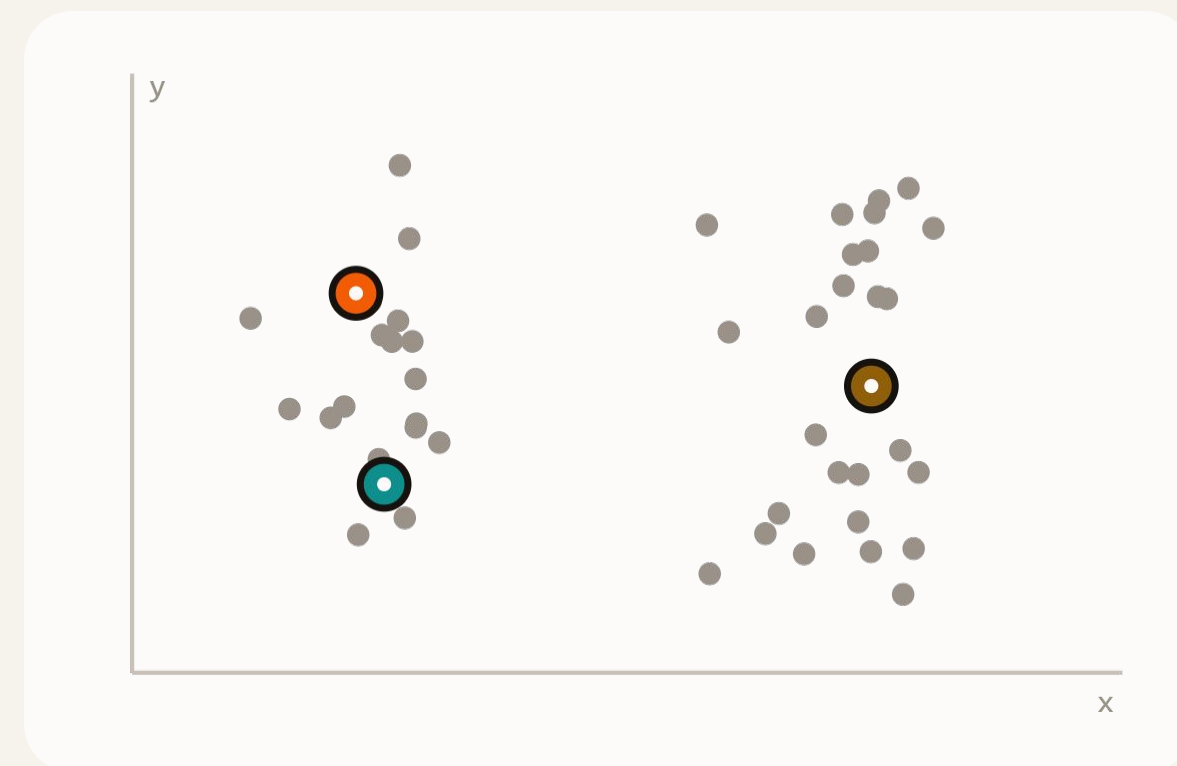
44 points. Our eyes see **3 groups** : one tall group on the left, two on the right.

With $K = 3$, this is the answer we want.

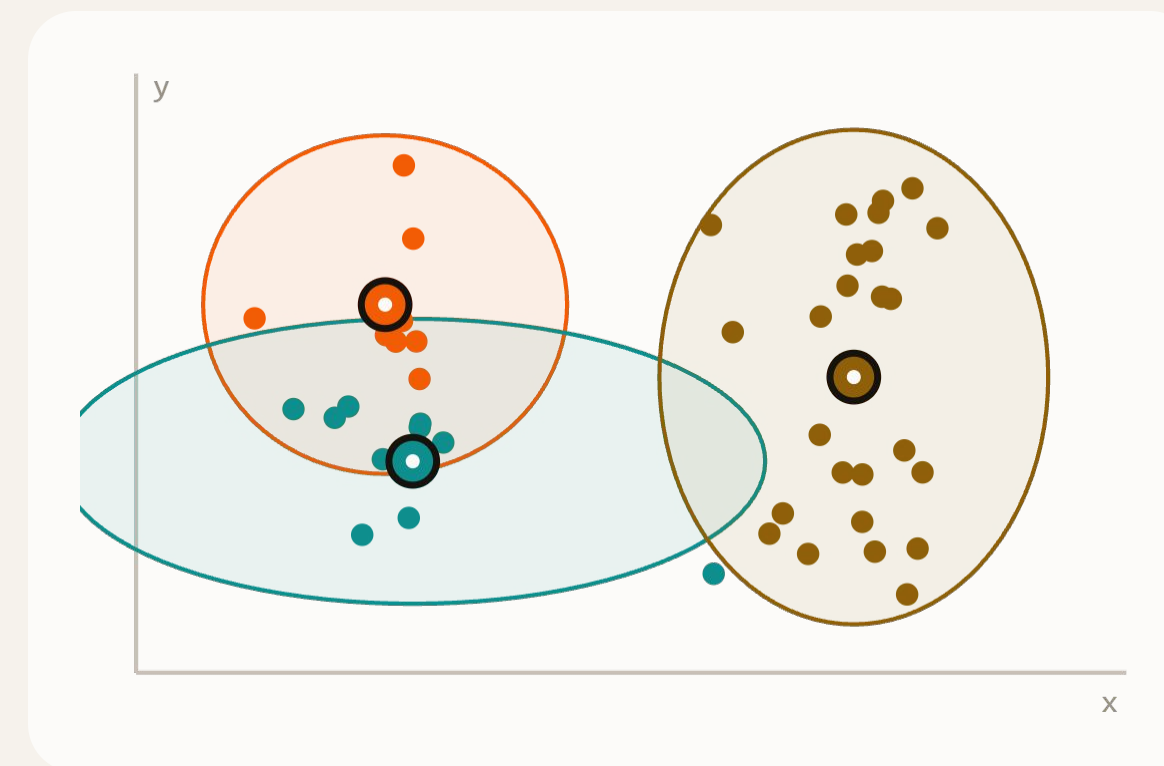
But step 1 places the centroids at **random** . What if the random spots are bad?

A Bad Random Start Gives a Wrong Answer

random start



where K-Means ends up



Two centroids started inside the left group. One started between the right groups.

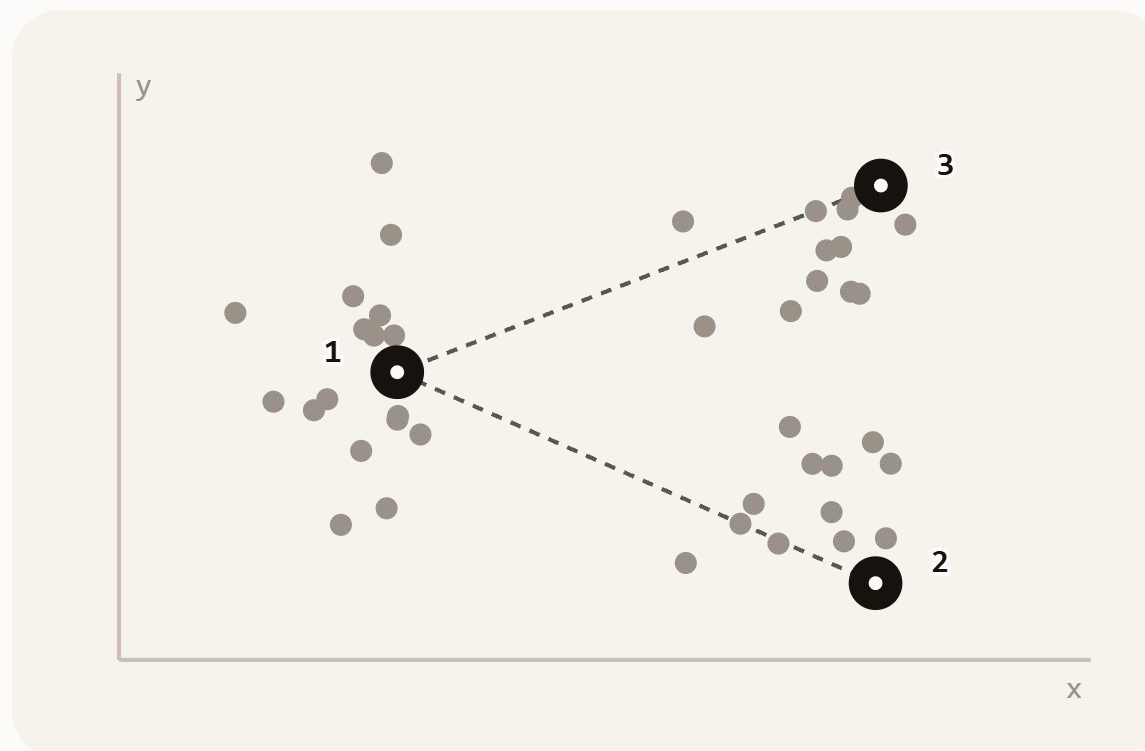
Left group: **cut in two** .
Right groups: **merged**.

WCSS 202.2, against 83.6 for the right answer.

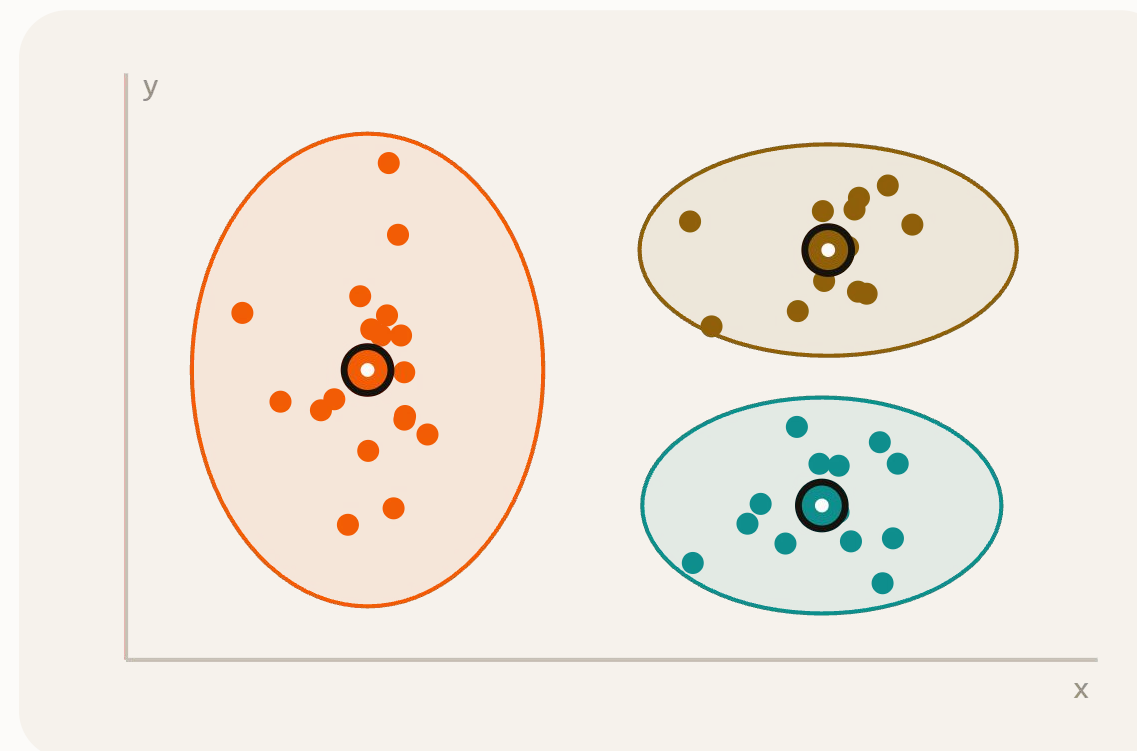
This is the **random initialisation trap** .

K-Means++: Start the Centroids Far Apart

K-Means++ start



where K-Means ends up



1. First centroid: a random data point.

2. Next centroid: far points get a much bigger chance to be picked.

3. Repeat until K centroids. They end up far apart.

scikit-learn already does this: `init='k-means++'`

RECAP

What to Carry Out of This Session

K-Means puts similar points into **K clusters** . Each cluster has a **centroid** at its centre.

Three steps: place centroids, join the nearest one, move to the average.
Repeat until no point changes group.

Choose K with **WCSS** and the **elbow method** : pick K where the curve bends and goes flat.

Euclidean distance is the straight line.
Manhattan goes along the grid.

A bad random start can give wrong clusters: the **random initialisation trap** .

K-Means++ starts the centroids far apart and avoids the trap.

NEXT

K-Means in the notebook

Then hierarchical clustering